

Class 4 Assignment

July 9, 2026

```
[2]: #We must first import the Pandas library. We will shortcut it to "pd".
import pandas as pd

# The following code will process the current filepath for the dataset

#This imports the necessary libraries
import os
from os.path import curdir

'''
The os.path.join function is used to merge strings (text) that correspond to
file and directory names together to a viable file path for your Operating
System. I printed the result to give more clarity.
'''

csv_path = os.path.join(curdir, 'Data', 'MMA_860_Grocery_Data.csv')
print(csv_path)
```

./Data/MMA_860_Grocery_Data.csv

```
[12]: '''
We repeat the above procedure, substituting the excel file name into the
join arguments and using the read_excel function instead.
'''

xl_path = os.path.join(curdir, 'Data', "MMA_860_Grocery_Data.csv")
df_xl = pd.read_csv(xl_path,
                    index_col=0)
df_xl.head()
```

```
[12]: Grocery_Bill  N_Adults  Family_Income  Family_Size  N_Vehicles  \
Obs
1      $357.73      2      $142,141      4      3
2      $276.84      2      $145,916      2      1
3      $197.92      1      $86,185      1      2
4      $315.75      2      $145,998      3      1
5      $202.89      1      $79,341      1      2

Distance_to_Store  Vegetarian  N_Children  Family_Pet
Obs
```

1	15	0	2	1
2	4	0	0	0
3	14	0	0	0
4	8	0	1	0
5	19	1	0	0

```
[14]: df_xl.dtypes
```

```
[14]: Grocery_Bill      str
      N_Adults        int64
      Family_Income    str
      Family_Size      int64
      N_Vehicles       int64
      Distance_to_Store int64
      Vegetarian       int64
      N_Children       int64
      Family_Pet       int64
      dtype: object
```

```
[15]: #First declare a function that removes a dollar sign from a string.
def remove_dollar_sign(s):
    '''
    The Python String replace function can be used to replace any
    instances of a substring in a string with another substring.
    In this case, we are replacing "$" with "" (an empty string);
    this effectively gets rid of any dollar signs in your string.
    '''
    return s.replace("$", "")
```

```
[16]: df_xl["Grocery_Bill"] = df_xl["Grocery_Bill"].apply(remove_dollar_sign)
```

```
[18]: df_xl.head()
```

```
[18]:   Grocery_Bill  N_Adults  Family_Income  Family_Size  N_Vehicles  \
Obs
1         357.73         2    $142,141           4           3
2         276.84         2    $145,916           2           1
3         197.92         1     $86,185           1           2
4         315.75         2    $145,998           3           1
5         202.89         1     $79,341           1           2

      Distance_to_Store  Vegetarian  N_Children  Family_Pet
Obs
1              15          0           2           1
2              4          0           0           0
3             14          0           0           0
4              8          0           1           0
```

5	19	1	0	0
---	----	---	---	---

```
[19]: df_xl["Family_Income"] = df_xl["Family_Income"].apply(remove_dollar_sign)
```

```
[20]: df_xl.head()
```

```
[20]:
```

	Grocery_Bill	N_Adults	Family_Income	Family_Size	N_Vehicles	\
Obs						
1	357.73	2	142,141	4	3	
2	276.84	2	145,916	2	1	
3	197.92	1	86,185	1	2	
4	315.75	2	145,998	3	1	
5	202.89	1	79,341	1	2	

	Distance_to_Store	Vegetarian	N_Children	Family_Pet
Obs				
1	15	0	2	1
2	4	0	0	0
3	14	0	0	0
4	8	0	1	0
5	19	1	0	0

```
[21]: '''
Family_Income contains a leading dollar sign as well as commas. We must
write an additional function to remove the commas. We can reuse the general
format from the dollar sign.
'''
def remove_comma(s):
    return s.replace(",","")
```

```
[22]: df_xl["Family_Income"] = df_xl["Family_Income"].apply(remove_comma)
```

```
[23]: df_xl.head()
```

```
[23]:
```

	Grocery_Bill	N_Adults	Family_Income	Family_Size	N_Vehicles	\
Obs						
1	357.73	2	142141	4	3	
2	276.84	2	145916	2	1	
3	197.92	1	86185	1	2	
4	315.75	2	145998	3	1	
5	202.89	1	79341	1	2	

	Distance_to_Store	Vegetarian	N_Children	Family_Pet
Obs				
1	15	0	2	1
2	4	0	0	0
3	14	0	0	0

4	8	0	1	0
5	19	1	0	0

```
[24]: df_xl = df_xl.astype({"Family_Income": float, "Grocery_Bill": float})
```

```
[25]: df_xl.dtypes
```

```
[25]: Grocery_Bill      float64
      N_Adults         int64
      Family_Income    float64
      Family_Size      int64
      N_Vehicles       int64
      Distance_to_Store int64
      Vegetarian        int64
      N_Children        int64
      Family_Pet        int64
      dtype: object
```

```
[26]: import statsmodels.api as sm
      from statsmodels.formula.api import ols
```

```
[29]: model = ols('Grocery_Bill ~ Family_Size + Family_Income + N_Adults + N_Children_
      ↪ + N_Vehicles + Distance_to_Store + Vegetarian + Family_Pet',df_xl).fit()
      print(model.summary())
```

```

                                OLS Regression Results
=====
Dep. Variable:          Grocery_Bill      R-squared:                0.839
Model:                  OLS              Adj. R-squared:           0.838
Method:                 Least Squares    F-statistic:                740.1
Date:                  Thu, 09 Jul 2026   Prob (F-statistic):         0.00
Time:                  18:33:08          Log-Likelihood:            -4900.2
No. Observations:      1000             AIC:                      9816.
Df Residuals:          992              BIC:                      9856.
Df Model:               7
Covariance Type:       nonrobust
=====
=====
coef      std err          t      P>|t|      [0.025
0.975]
-----
-----
Intercept          23.3057      5.715      4.078      0.000      12.091
34.520
Family_Size        27.6127      1.277     21.619      0.000      25.106
30.119
Family_Income       0.0009   8.77e-05     10.762      0.000      0.001
0.001

```

N_Adults	26.8743	2.637	10.193	0.000	21.700
32.048					
N_Children	0.7384	1.608	0.459	0.646	-2.416
3.893					
N_Vehicles	-0.8388	1.149	-0.730	0.465	-3.093
1.416					
Distance_to_Store	3.8897	0.191	20.391	0.000	3.515
4.264					
Vegetarian	-7.8462	4.254	-1.844	0.065	-16.194
0.501					
Family_Pet	1.6358	2.827	0.579	0.563	-3.913
7.184					

Omnibus:	51.552	Durbin-Watson:	2.096
Prob(Omnibus):	0.000	Jarque-Bera (JB):	172.112
Skew:	0.081	Prob(JB):	4.23e-38
Kurtosis:	5.026	Cond. No.	3.67e+20

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The smallest eigenvalue is 8.53e-29. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

```
[30]: # Getting rid of vehicles first since bad p value with t stat. Seems not too
      ↪significant

model = ols('Grocery_Bill ~ Family_Size + Family_Income + N_Adults + N_Children_
      ↪ + Distance_to_Store + Vegetarian + Family_Pet',df_xl).fit()
print(model.summary())
```

OLS Regression Results

Dep. Variable:	Grocery_Bill	R-squared:	0.839
Model:	OLS	Adj. R-squared:	0.838
Method:	Least Squares	F-statistic:	863.7
Date:	Thu, 09 Jul 2026	Prob (F-statistic):	0.00
Time:	18:37:14	Log-Likelihood:	-4900.4
No. Observations:	1000	AIC:	9815.
Df Residuals:	993	BIC:	9849.
Df Model:	6		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025
0.975]					

```

-----
Intercept          21.7000      5.273      4.115      0.000      11.352
32.048
Family_Size        27.5776      1.276      21.612      0.000      25.074
30.082
Family_Income       0.0009      8.77e-05     10.778      0.000      0.001
0.001
N_Adults           26.8361      2.635      10.183      0.000      21.664
32.008
N_Children          0.7416      1.607      0.461      0.645      -2.413
3.896
Distance_to_Store   3.8913      0.191      20.405      0.000      3.517
4.266
Vegetarian          -7.7810      4.252      -1.830      0.068      -16.125
0.563
Family_Pet          1.7474      2.823      0.619      0.536      -3.792
7.287

=====
Omnibus:              52.124    Durbin-Watson:              2.097
Prob(Omnibus):         0.000    Jarque-Bera (JB):          175.903
Skew:                  0.079    Prob(JB):                  6.35e-39
Kurtosis:              5.049    Cond. No.                  1.78e+20
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The smallest eigenvalue is 3.63e-28. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

```

[32]: # Children seems to be bad data as well, however, we will instead remove family_
      ↪size first since we have breakdown of children and adults and they might_
      ↪have different spending habits

model = ols('Grocery_Bill ~ Family_Income + N_Adults + N_Children +_
      ↪Distance_to_Store + Vegetarian + Family_Pet',df_xl).fit()
print(model.summary())

```

OLS Regression Results

```

=====
Dep. Variable:          Grocery_Bill    R-squared:              0.839
Model:                  OLS             Adj. R-squared:        0.838
Method:                 Least Squares    F-statistic:           863.7
Date:                   Thu, 09 Jul 2026  Prob (F-statistic):    0.00
Time:                   18:39:11         Log-Likelihood:        -4900.4
No. Observations:       1000            AIC:                  9815.
Df Residuals:           993            BIC:                  9849.
Df Model:                6

```

```

Covariance Type:          nonrobust
=====
=====
              coef      std err          t      P>|t|      [0.025
0.975]
-----
Intercept          21.7000      5.273      4.115      0.000      11.352
32.048
Family_Income       0.0009   8.77e-05     10.778      0.000       0.001
0.001
N_Adults           54.4137      3.816     14.258      0.000      46.925
61.903
N_Children         28.3192      1.216     23.296      0.000      25.934
30.705
Distance_to_Store   3.8913      0.191     20.405      0.000       3.517
4.266
Vegetarian         -7.7810      4.252     -1.830     0.068     -16.125
0.563
Family_Pet          1.7474      2.823      0.619     0.536      -3.792
7.287
=====
Omnibus:              52.124   Durbin-Watson:              2.097
Prob(Omnibus):         0.000   Jarque-Bera (JB):         175.903
Skew:                  0.079   Prob(JB):                 6.35e-39
Kurtosis:              5.049   Cond. No.                 5.81e+05
=====

```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 5.81e+05. This might indicate that there are strong multicollinearity or other numerical problems.

[35]: *# Pet does not influence grocery bill. P value with T stat is different*

```

model = ols('Grocery_Bill ~ Family_Income + N_Adults + N_Children +_
↪Distance_to_Store + Vegetarian',df_xl).fit()
print(model.summary())

```

OLS Regression Results

```

=====
Dep. Variable:          Grocery_Bill   R-squared:              0.839
Model:                  OLS           Adj. R-squared:        0.838
Method:                 Least Squares  F-statistic:           1037.
Date:                   Thu, 09 Jul 2026  Prob (F-statistic):      0.00
Time:                   18:41:34        Log-Likelihood:       -4900.6
No. Observations:      1000           AIC:                  9813.

```

```

Df Residuals:          994    BIC:          9843.
Df Model:              5
Covariance Type:      nonrobust

```

```

=====
=====
              coef      std err          t      P>|t|      [0.025
0.975]
-----
-----
Intercept      22.0447      5.242      4.205      0.000      11.758
32.332
Family_Income    0.0009    8.76e-05    10.773      0.000      0.001
0.001
N_Adults       54.4312      3.815     14.267      0.000      46.945
61.918
N_Children     28.3297      1.215     23.314      0.000      25.945
30.714
Distance_to_Store 3.8896      0.191     20.405      0.000      3.516
4.264
Vegetarian     -7.8390      4.250     -1.845     0.065     -16.178
0.500
=====
Omnibus:          52.446    Durbin-Watson:          2.094
Prob(Omnibus):    0.000    Jarque-Bera (JB):        177.774
Skew:             0.080    Prob(JB):                2.49e-39
Kurtosis:         5.059    Cond. No.                5.78e+05
=====

```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 5.78e+05. This might indicate that there are strong multicollinearity or other numerical problems.

[]: